YaKa 3.0 固件总体架构

当前小智 v2.5.2 / ESP-IDF 6.1 基线 + PH2–PH5 目标层

产品、交互与外部系统

产品合同
.agents/skills/yaka-project/references/
yaka-3.0-prd.md

用户输入与反馈
DPAD / ACT_A / ACT_B / PWR_A/B
屏幕 / RGB / 灯阵 / 振动 / 提示音

需求与验收

小智云端 / 业务后端
OTA config / IM 产品事件 WebSocket
STT / LLM / TTS / MCP

用户数据与本地介质
TF 录音 / 文件索引 / 恢复结果
NVS 凭据 / 设置 / 迁移

产品门禁
TBD 不由驱动冻结
需求 ID → 固件 → 测试证据

会话/音频/消息

控制面与产品状态

Application 单例
main/application.h|cc
主事件循环、协议生命周期、音频会话、OTA 触发
Application::Schedule() 回到主任务

TransitionTo 领域命令/事件

DeviceStateMachine
main/device_state.h
main/device_state_machine.h|cc
校验 starting / idle / listening / speaking / upgrading 等转换

ProductController（计划）
main/yaka/product_controller.*
唯一产品状态写入者
PowerPhase / ForegroundMode / Connectivity / CapabilityHealth

上游事件/命令

XiaozhiAdapter（计划）
main/yaka/xiaozhi_adapter.*
把上游 DeviceState/Protocol/Audio 事件映射为产品事件
防止 YaKa 逻辑散落到 application.cc

MCP 工具分发
main/mcp_server.h|cc
经 Protocol 传输；非 PH2 核心阻断项

控制/回调

生命周期/音频包

复用服务与 YaKa 领域服务

AudioService
main/audio/audio_service.*
AudioInputTask / AudioOutputTask / OpusCodecTask
LiteAudioEngine + FixedQueue

Protocol
main/protocols/protocol.*
mqtt_protocol.* / websocket_protocol.*
音频包、JSON、会话回调

WifiBoard / Wi-Fi manager
main/boards/common/wifi_board.*
配网、连接、网络事件、节能档位

领域命令/健康

平台通用服务
main/ota.* / settings.* / assets.*
system_info.* / notify/notify_player.*
display/* / led/*

Board 接口

窄适配

核心领域服务（计划）
main/yaka/dialogue_service.*
recorder_service.* / storage_service.*
input_service.* / feedback_service.*

辅助领域服务（计划）
main/yaka/nfc_service.* / power_service.*
update_service.* / diagnostics_service.*
capability_registry.* / error_codes.*

服务层规则
有界队列 / 超时 / 降级 / 稳定错误码
耗时 I/O 不进入主事件循环
I²S RX 单一所有者

Board / BSP / 驱动与能力健康

Board → WifiBoard
main/boards/common/board.*
main/boards/common/wifi_board.*
抽象 Display / Led / AudioCodec / Network / Battery

继承/工厂

ChatChat HiYaKa 板（M1 软件骨架）
main/boards/chatchat/hi-yaka/
hi_yaka_board.cc / config.h / config.json
启动诊断、I²C 探测、PCA9557 安全态
当前：DummyAudioCodec + NoDisplay + NoLed

逐阶段替换 Dummy

YaKa 驱动接入（计划）
ST7789 / ADC DPAD / ES7210 + ES8311
TF(SDSPI) / PN532 / QMI8658A
RGB / 五灯阵 / ERM / BAT ADC

结果/错误码

Capability Registry（计划）
unavailable / initializing / ready
degraded / failed
驱动结果 → 诊断/UI/动作门禁同一来源

实板证据门禁
L0 构建 ≠ L1 启动
驱动成功 ≠ L2 功能闭环
发布必须 L3 场景/长稳

物理 I/O

ESP-IDF 6.1 与 YaKa 3.0 硬件

ESP-IDF / FreeRTOS
任务、EventGroup、I²C、I²S、SPI、SDSPI
NVS、OTA、HTTP、Wi-Fi、RMT、ADC、LVGL

ESP32-C6
16 MiB DIO 驱动 API无 PSRAM
内部 SRAM 必须有预算和峰值门禁

共享 I²C0 GPIO10/11
ES8311 0x18 / PCA9557 0x19
PN532 0x24 / ES7210 0x41 / QMI8658A 0x6A

SPI / I²S / GPIO
ST7789 200×320 / TF SDSPI
双麦 + 播放参考 / PA + Speaker
DPAD / RGB / 灯阵 / 马达 / BAT ADC

电源边界
MASTER_SW 可任意硬切
文件、NVS、OTA 必须恢复
MCU 不可可靠观测完整充电终态

统一图例

蓝：小智/上游现有实现
绿：HiYaKa 本地已实现
橙虚线：待批准/待实现
黄：已确认合同/配置/产物
紫：外部系统/发布/协作副本
灰：物理对象/上下文/容器
红：约束/风险/门禁/证据
线型：实线=当前/已确认；虚线=计划关系

运行时控制面、事件汇聚与 FreeRTOS 任务

实线为当前/已确认关系；虚线为计划关系

启动顺序

app_main()
main/main.cc
NVS init / 必要时擦除

HiYaKa Board Factory
Board::GetInstance() → HiYaKaBoard
Flash/heap 日志 → I²C → PCA9557 安全态 → 五地址探测

Application::Initialize()
main/application.cc
Display::SetupUI → AudioService::Initialize/Start
注册音频/状态/网络回调 → StartNetwork

Application::Run()
主任务优先级 10；永不返回
xEventGroupWaitBits → 串行处理事件

activation task
栈 8192 / 优先级 2
Assets → OTA check/activation → Protocol init

计划：ProductController 启动
读取 CapabilityHealth
决定配网 / 首页 / 离线 / 恢复
不绕过 Application 和 DeviceStateMachine

启动安全约束
PA、背光、RGB、马达默认关闭
单个非关键外设失败必须降级
不得用重启循环掩盖失败

主任务事件汇聚

Application EventGroup
MAIN_EVENT_SCHEDULE / SEND_AUDIO / WAKE_WORD / VAD / ERROR
NETWORK_CONNECTED / DISCONNECTED / TOGGLE_CHAT / START/STOP_LISTENING
STATE_CHANGED / PLAYBACK_DRAINED / CLOCK_TICK

dispatch

主事件循环：唯一上游应用变更上下文
错误 → 网络 → 激活 → 状态变化 → 播放排空 → 用户动作 → 音频发送 → 定时
状态改变统一调用 Application::SetDeviceState()

MAIN_EVENT_SCHEDULE

Application::Schedule()
mutex _ + main_tasks_ (std::deque)
跨任务回调先排队，再由 MAIN_EVENT_SCHEDULE 串行执行
PH2 需定义容量/拒绝策略，避免无界积压

计划：固定队列

wait bits

DeviceStateMachine
atomic current_state + 转换表 + listener
listener 只置 MAIN_EVENT_STATE_CHANGED；UI/LED/Audio 在主任务响应

计划：ProductEventQueue
固定容量；事件带来源、时间戳、关联 ID、超时和丢弃指标
ISR/驱动/后台任务只投递事件，ProductController 串行决定业务状态

主任务输出
Display / Led / Feedback、Protocol 命令、AudioService 控制
耗时网络、文件和 I²C 操作必须交给服务任务

并发门禁
不在回调里直接改 UI/产品状态；不阻塞 main/audio；所有队列有容量和指标
当前 main_tasks_ 为无界 deque，是 PH2 需要显式治理的现有风险

send/wake/VAD/drained

network/protocol callbacks

event bits

Schedule

后台任务与数据任务

audio_input
优先级 8；C6 栈 4096
AudioCodec::InputData → LiteAudioEngine
采集任务不能写文件/发网络

opus_codec
优先级 2；栈 24576
encode/decode FixedQueue
send/decode/playback 有界

audio_output
优先级 4；C6 栈 2048
播放队列 → AudioCodec::OutputData
播放排空回调通知主任务

网络/协议内部任务
Wi-Fi manager / MQTT / WebSocket / HTTP
回调只置事件位或 Schedule
共享 Protocol 语义需双传输验证

PCM 块引用/复制

notify_http
优先级 2；栈 6144
HTTP 拉取媒体 → AudioService 解码队列
字幕/完成回调经 Schedule

esp_timer 回调
clock_timer / audio_power_timer
wifi connect timer / MQTT reconnect
只发通知，不做耗时工作

计划：recorder_task
固定块池 + 有界写盘队列
.part → checkpoint → fsync → 原子完成
永不阻塞 audio_input

计划：housekeeping
BAT ADC / NFC / IMU 低频轮询
能力健康与资源指标
有超时、退避和取消

健康/传感事件

LVGL/显示上下文
局部缓冲；DisplayLockGuard
只渲染状态，不拥有业务真值
当前 YaKa 仍为 NoDisplay

资源观测
heap min / largest block / stack high-water
queue peak / drop / audio underrun
PH4 进入 CI 门禁

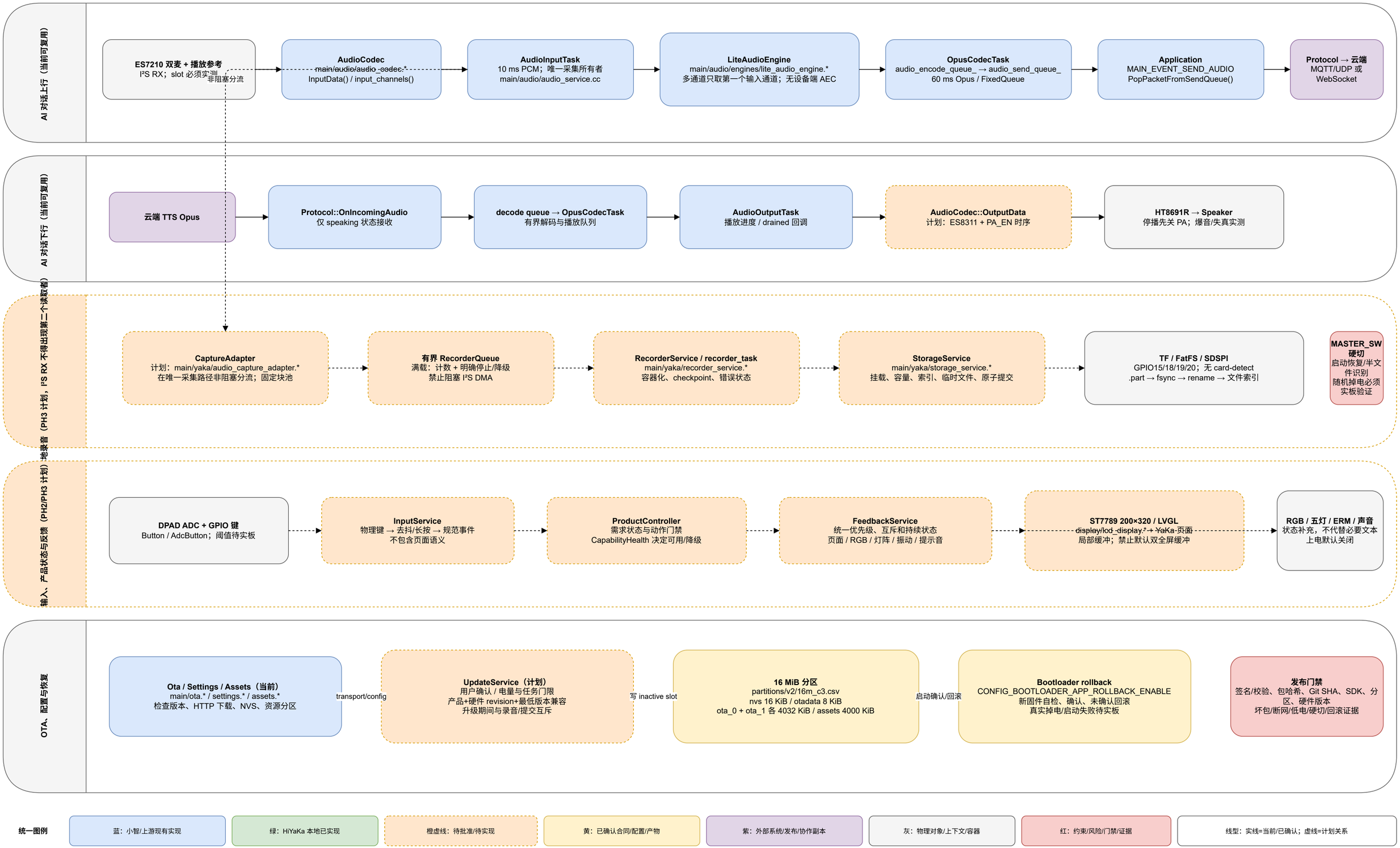
C6 无 PSRAM结论
不能靠大缓存掩盖背压问题；音频、UI、网络、TF 并发必须以内部 SRAM 峰值成立
任何计划任务的栈和队列容量均需预算

统一图例

- 蓝：小智/上游现有实现
- 绿：HiYaKa 本地已实现
- 橙虚线：待批准/待实现
- 黄：已确认合同/配置/产物
- 紫：外部系统/发布/协作副本
- 灰：物理对象/上下文/容器
- 红：约束/风险/门禁/证据
- 线型：实线=当前/已确认；虚线=计划关系

关键数据路径与所有权

展示当前上游链路以及 YaKa 本地录音、产品 UI、OTA 策略的计划接缝



构建选择链、上游同步、CI 与三端文档治理

唯一板型构建选择链

ChatChat HiYaKa 构建入口
python scripts/build.py
board ID：chatchat/hi-yaka
产物 ID：hi-yaka

→ **board config.json**
target=esp32c6
DIO / wake word off / device AEC off

→ **sdkconfig defaults 合并**
sdkconfig.defaults
sdkconfig.defaults.esp32c6
board sdkconfig_append
最终：16 MiB / 16m_c3.csv / DIO

→ **Kconfig 选择**
main/Kconfig.projbuild
宏：BOARD_TYPE_HI_YAKA
depends on IDF_TARGET_ESP32C6

→ **CMake 源选择**
main/CMakeLists.txt
BOARD_DIR=chatchat/hi-yaka
字体、图标、AudioEngine、依赖组件

→ **HiYaKa 单一 Board Factory**
文件：hi_yaka_board.cc
类：HiYaKaBoard
每个固件只能有一个工厂

分区与资源
partitions/v2/16m_c3.csv
assets pipeline / fonts / locale / SPIFFS image

→ **可追溯产物**
bootloader.bin / partition-table.bin / xiaozhi.bin / assets.bin / merged image
版本、上游 SHA、SDK、sdkconfig、分区与 SHA-256

仓库与上游策略

upstream：78/xiaozhi-esp32
只拉取；当前固定 v2.5.2 / 49b4380
不追随 upstream/main

merge release tag

origin：njueeRay/hi-yaka-firmware
私有开发真值；完整历史、PR、CI、内部文档
main / feature/* / sync/xiaozhi-vX.Y.Z

独立上游同步 PR
git fetch upstream --tags → sync 分支 → merge release tag
不混入产品功能；构建 HiYaKa C6 + 上游 C6 代表板

受控快照

公开快照：ChatChatTech/yaka/firmware
只发布审查后的受控快照；不镜像私有历史/内部 PRD/凭据
记录私有来源 SHA、上游 SHA、SDK、验证等级与排除范围

CI、Raynode 与验证等级

HiYaKa board contract
.github/workflows/yaka-contract.yml
scripts/tests/test_hi_yaka_board.py
板型、选择链、无 PSRAM、PCA 顺序、未启用外设

相同 Git SHA

Raynode 可复现构建
/srv/projects/hi-yaka-firmware
espressif/idf:v6.1 固定 digest
远程构建、产物与日志

上游 CI/host tests
.github/workflows/build.yml
python -m unittest discover -s scripts/tests -v
代表 target / network / assets / release 检查

证据等级
L0：静态/host/build/hash
L1：实板启动/总线/驱动
L2：功能/故障注入/重复性
L3：整机场景/功耗/声学/长稳

PH0–PH5 只能关闭 L0 项；M1–M7 涉及物理行为的 checklist 必须等待 L1–L3。构建成功、模拟成功和驱动返回成功都不能写成实板通过。

本地 → GitHub → 飞书的文档同步

产品 PRD
.agents/skills/yaka-project/references/
yaka-3.0-prd.md
用户/场景/范围/交互/产品验收

固件 PRD 与里程碑
~~docs/governance~~提交~~IRMWARE_PRD.md~~
YFWR、PH0–PH5、M1–M7、DoD

硬件连接与接口表
docs/hardware/HARDWARE_MODULE_INTERFACES.md
GPIO / 总线 / 地址 / 极性 / 初始化 / 首板矩阵

架构图与说明
docs/yaka/firmware-architecture.md
docs/yaka/diagrams/yaka-firmware-architecture.drawio
代码层、运行时、数据流、构建治理

同步登记
docs/governance/FEISHU_SYNC_REGISTRY.md
token / revision / hash / 差异状态

提交　提交

GitHub 私有仓库
docs/README.md 提供三份核心文档阅读顺序
提交 SHA 是本地 Markdown 版本身份

同步正文/附件

飞书协作副本
01 产品需求与交互状态 / 01 固件 PRD 与里程碑
03 硬件连接与固件接口表 / 硬件主文档附件入口
同步后必须回读关键章节和 revision

阶段关闭规则
~~代码/文档/证据 → 本地~~回读 revision~~ → GitHub 推送 → 飞书同步~~ → revision 回写
任一步缺失都不能把阶段标为已完成

统一图例

蓝：小智/上游现有实现
绿：HiYaKa 本地已实现
橙虚线：待批准/待实现
黄：已确认合同/配置/产物
紫：外部系统/发布/协作副本
灰：物理对象/上下文/容器
红：约束/风险/门禁/证据
线型：实线=当前/已确认；虚线=计划关系